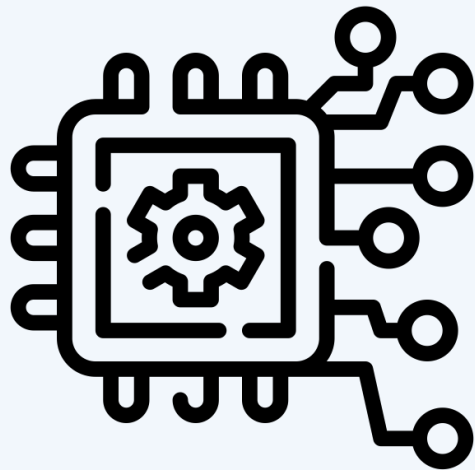
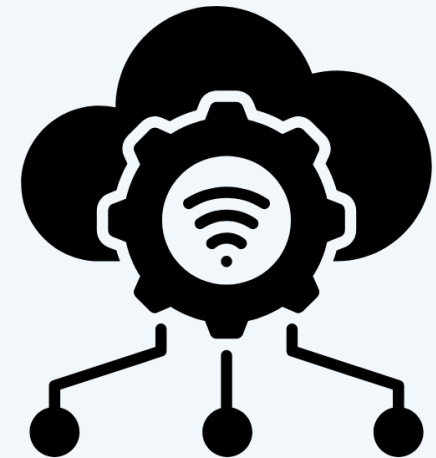


Internet of Things – Unit 2



Dr. Abhishek Joshi
ICT, PDEU



Unit 2 - Technology for IoT

- Programming for IoT
- Different frameworks
- Virtualization concepts
- Embedded platforms
- Cloud-based deployment
- Distributed data analysis

IoT Programming - Overview

- Why Multiple Languages?
 - Different IoT devices have different capabilities:
 - Microcontrollers → Limited resources (KB of RAM)
 - Single Board Computers → Full OS, GB of RAM
 - Cloud/Edge Gateways → High performance

IoT Programming - Overview

- IoT Programming Layers:
- **Application Logic**: Python, JavaScript, Node.js
 - (Business rules, UI, data processing)
- **Device Firmware**: C, C++, Rust, MicroPython
 - (Hardware control, sensors, actuators)
- **Cloud/Gateway**: Python, JavaScript, Go
 - (Data aggregation, analytics, APIs)

IoT Programming - Overview

Language	Primary Use	Why Chosen	Examples
C	Low-level firmware on microcontrollers	Efficient, direct hardware control, minimal overhead	Arduino Uno, ESP8266
C++	Mid-level firmware, complex IoT devices	Object-oriented, performance	ESP32, STM32
Python	Edge computing, Raspberry Pi, data processing	Simple syntax, rich libraries	Raspberry Pi, Gateways
MicroPython	Lightweight Python for microcontrollers	Python on MCUs, easy prototyping	ESP32, Raspberry Pi Pico
JavaScript/Node.js	IoT dashboards, server-side	Web integration, real-time updates	Dashboards, APIs

Language Selection Guide

- Resource - Constrained (KB RAM) → C / C++ / MicroPython
 - (Arduino, ESP8266, small sensors)
- Balanced (MB RAM, Wi-Fi/BT) → C++ / MicroPython
 - (ESP32, Raspberry Pi Pico)
- Powerful (GB RAM, Full OS) → Python / JavaScript
 - (Raspberry Pi, Gateways, Cloud)

Language Selection Guide

- Resource - Constrained (KB RAM) → C / C++ / MicroPython
 - (Arduino, ESP8266, small sensors)
- Balanced (MB RAM, Wi-Fi/BT) → C++ / MicroPython
 - (ESP32, Raspberry Pi Pico)
- Powerful (GB RAM, Full OS) → Python / JavaScript
 - (Raspberry Pi, Gateways, Cloud)

Language Selection Guide

- 2026 Trends:
 - Rust gaining traction for safety-critical IoT
 - MicroPython adoption increasing[
 - Professionals favor C++ and C

Programming Example - Arduino (C++)

- Blinking LED - Arduino (C/C++)

```
void setup() {  
    pinMode(13, OUTPUT);  
}  
  
void loop() {  
    digitalWrite(13, HIGH);  
    delay(1000);  
    digitalWrite(13, LOW);  
    delay(1000);  
}
```

Key Concepts:

- `setup()` → Runs once at startup
- `loop()` → Runs continuously
- `pinMode()` → Configure pin as input/output
- `digitalWrite()` → Set pin HIGH/LOW

Programming Example - Arduino (C++)

- Blinking LED - MicroPython on ESP32

```
from machine import Pin
import time
led = Pin(2, Pin.OUT)      # Onboard LED
on ESP32
while True:
    led.value(1)            # Turn LED ON
    time.sleep(1)          # Wait 1 second
    led.value(0)            # Turn LED OFF
    time.sleep(1)          # Wait 1 second
```

Why MicroPython?

- Python syntax on microcontrollers
- Easier for beginners
- Supports ESP32, Raspberry Pi Pico, PyBoard

IoT Framework - Overview

- IoT framework is essentially a comprehensive toolkit that provides developers with the building blocks needed to create, deploy, and manage Internet of Things applications
- Instead of building everything from scratch, developers can leverage these frameworks to handle the complex aspects of IoT development such as device connectivity, data management, and security

IoT Framework - Overview

- A collection of integrated tools, services, APIs, and libraries designed specifically for IoT application development
- It accelerates development by providing pre-built components
- Device connectivity, data ingestion, storage, analytics, device management, security, and over-the-air updates
- The primary goal of an IoT framework is to reduce complexity and time-to-market while ensuring that the application is scalable, secure, and reliable

IoT Frameworks - Categories

- Cloud-Based (Platform as a Service), Open-Source (Self-Hosted)

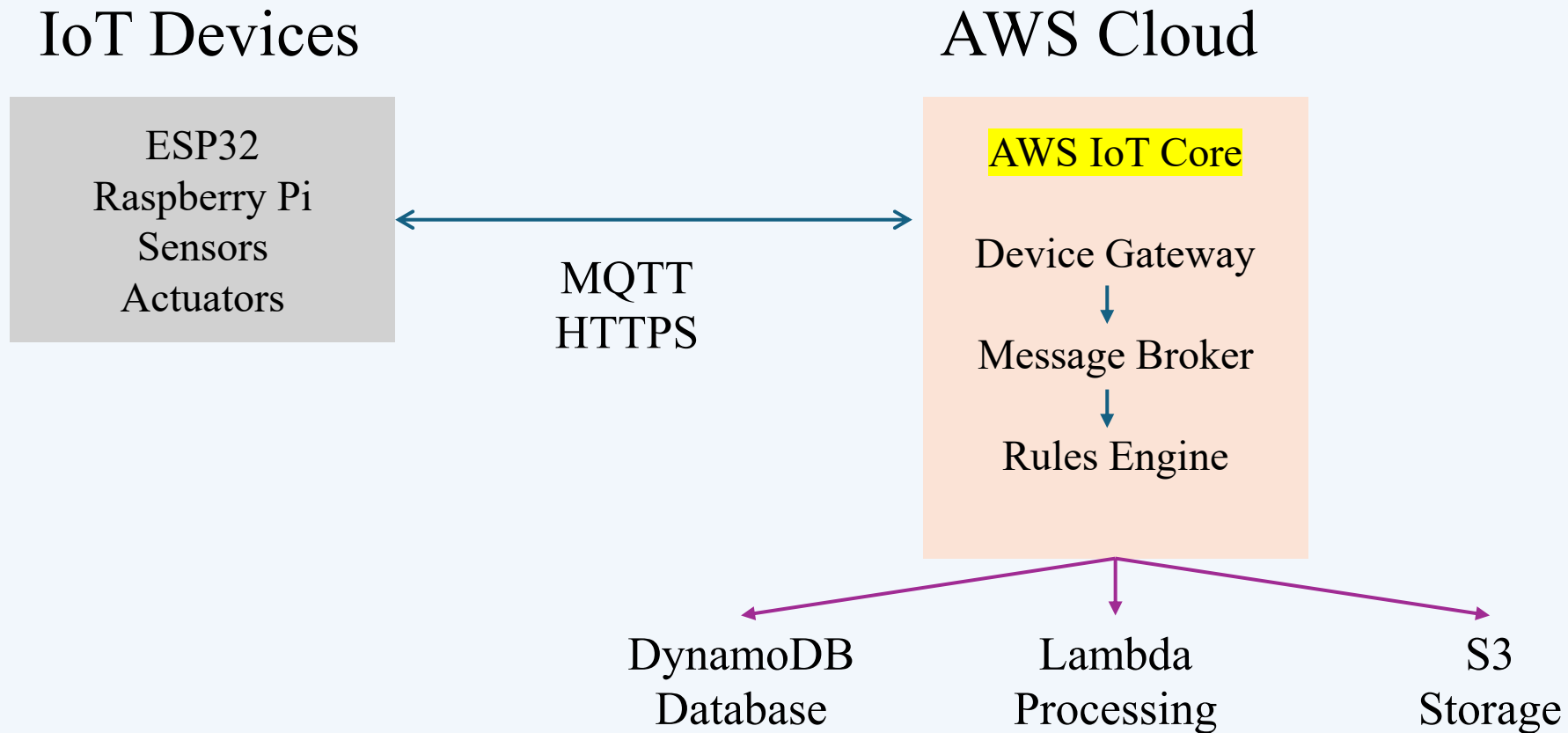
Aspect	Cloud-Based (PaaS)	Open-Source (Self-Hosted)
Hosting	Provider-managed	Self-managed
Maintenance	Minimal; automatic updates & scaling	Full responsibility
Scalability	Automatic / auto-scaling	Manual planning required
Cost	Pay-as-you-go	Free software + hosting cost
Vendor Lock-in	High	Low
Customization	Limited	Extensive
Best For	Rapid deployment, enterprises	Full control, customization & compliance

IoT Frameworks - AWS IoT Core

- Managed cloud service provided by Amazon Web Services
- Enables IoT devices to securely connect to the cloud
- Supports device-to-cloud and cloud-to-device communication
- Provides communication, security, device management, and integration with AWS services

AWS IoT Core acts as the communication and management layer between IoT devices and cloud applications

IoT Frameworks - AWS IoT Core



IoT Frameworks - AWS IoT Core

- AWS IoT Core allows devices such as:

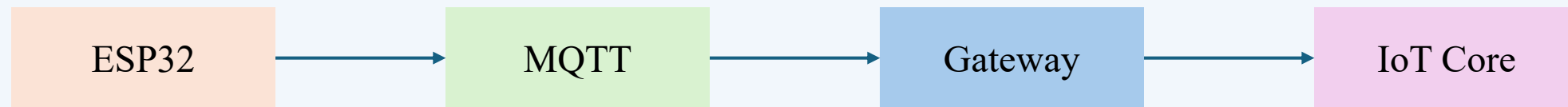
- ESP32
- Raspberry Pi
- Industrial controllers
- Smart meters
- Sensors and actuators
- Connected vehicles

Communication protocols:

- MQTT
- MQTT over WebSocket
- HTTPS

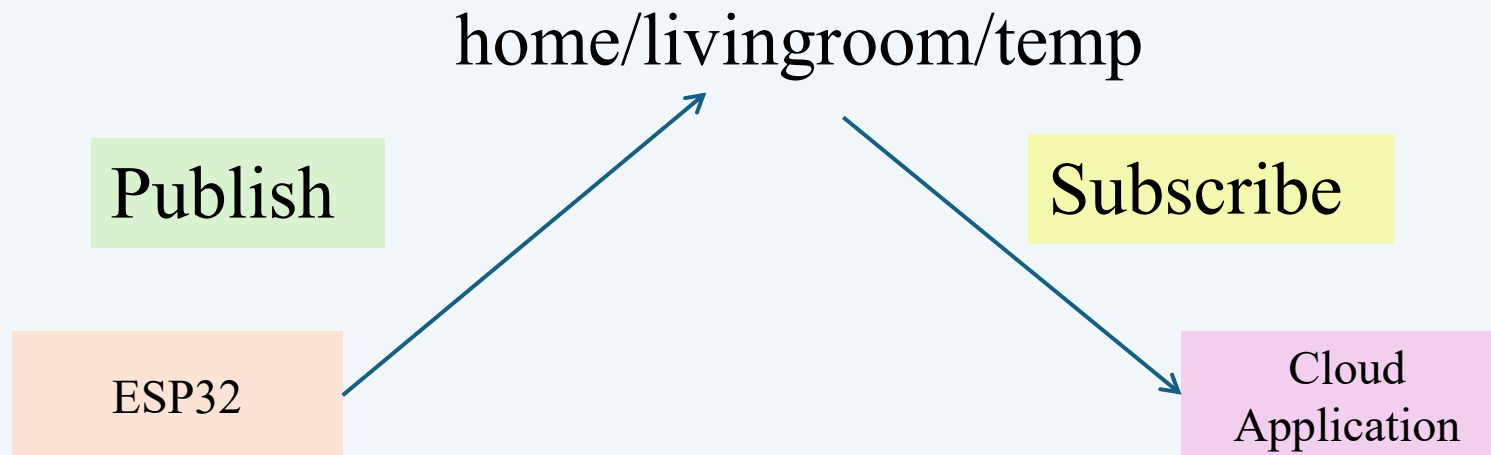
IoT Frameworks - AWS IoT Core

- Device Gateway is the entry point for IoT devices connecting to AWS IoT Core
 - Accept device connections
 - Handle MQTT/HTTPS communication
 - Maintain secure connections
 - Support large numbers of connected devices



IoT Frameworks - AWS IoT Core

- Message Broker enables communication using the publish/subscribe model



ESP32 publishes

Topic: home/temperature

Payload:

```
{
    "temperature":
28.5,
    "humidity": 67
}
```

IoT Frameworks - AWS IoT Core

- AWS IoT Core provides strong device-level security
 - X.509 certificates
 - TLS encryption
 - IoT policies
 - Authentication
 - Authorization
 - IAM integration

A device can be authorized to:

Publish:

factory/device01/temperature

Subscribe:

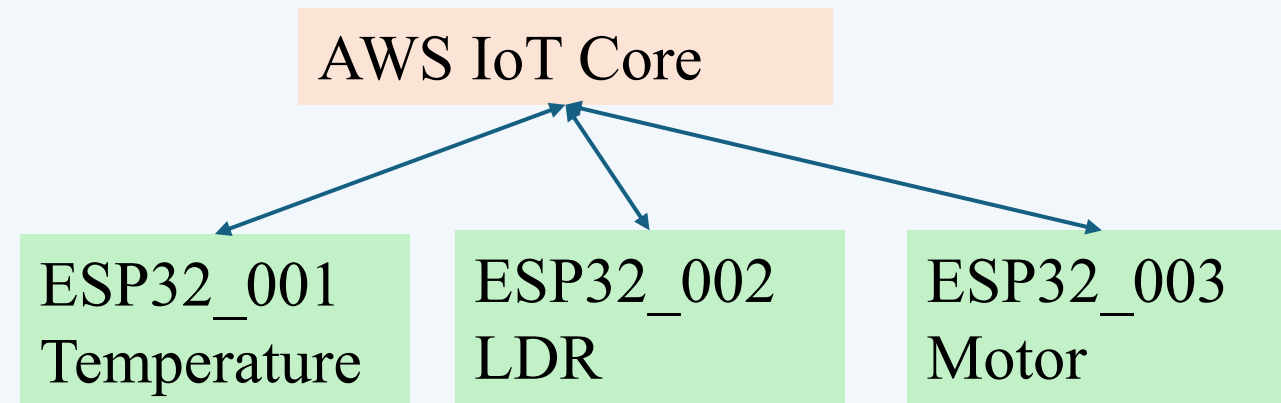
factory/device01/command

but denied access to another device's topics

IoT Frameworks - AWS IoT Core

- The Device Registry maintains information about IoT devices

Device ID	: ESP32_001
Device Type	: Temperature
SensorLocation	: Laboratory
Firmware	: v2.1
Status	: Active



- To uniquely identify and organize devices

IoT Frameworks - AWS IoT Core

- A Device Shadow is a cloud-side representation of the state of a physical device

```
{ "desired": {  
    "power": "ON",  
    "speed": 3  
  },  
  "reported": {  
    "power": "OFF",  
    "speed": 2  
  }  
}
```

- **Desired state** → what the application wants
- **Reported state** → current physical device state

State can be synchronized even when a device temporarily loses connectivity

IoT Frameworks - AWS IoT Core

- The Rules Engine processes incoming IoT messages and routes them to other AWS services

IF temperature $> 80^{\circ}\text{C}$



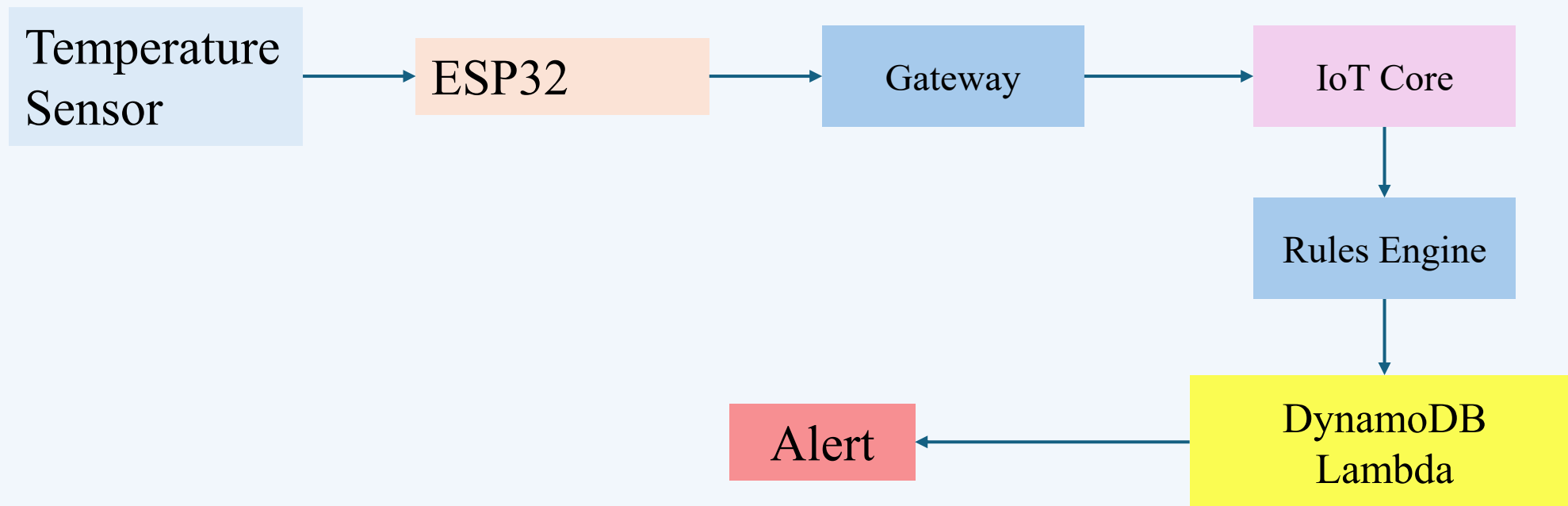
Trigger Lambda



Generate Alert

IoT Frameworks - AWS IoT Core

- Industrial Temperature Monitoring

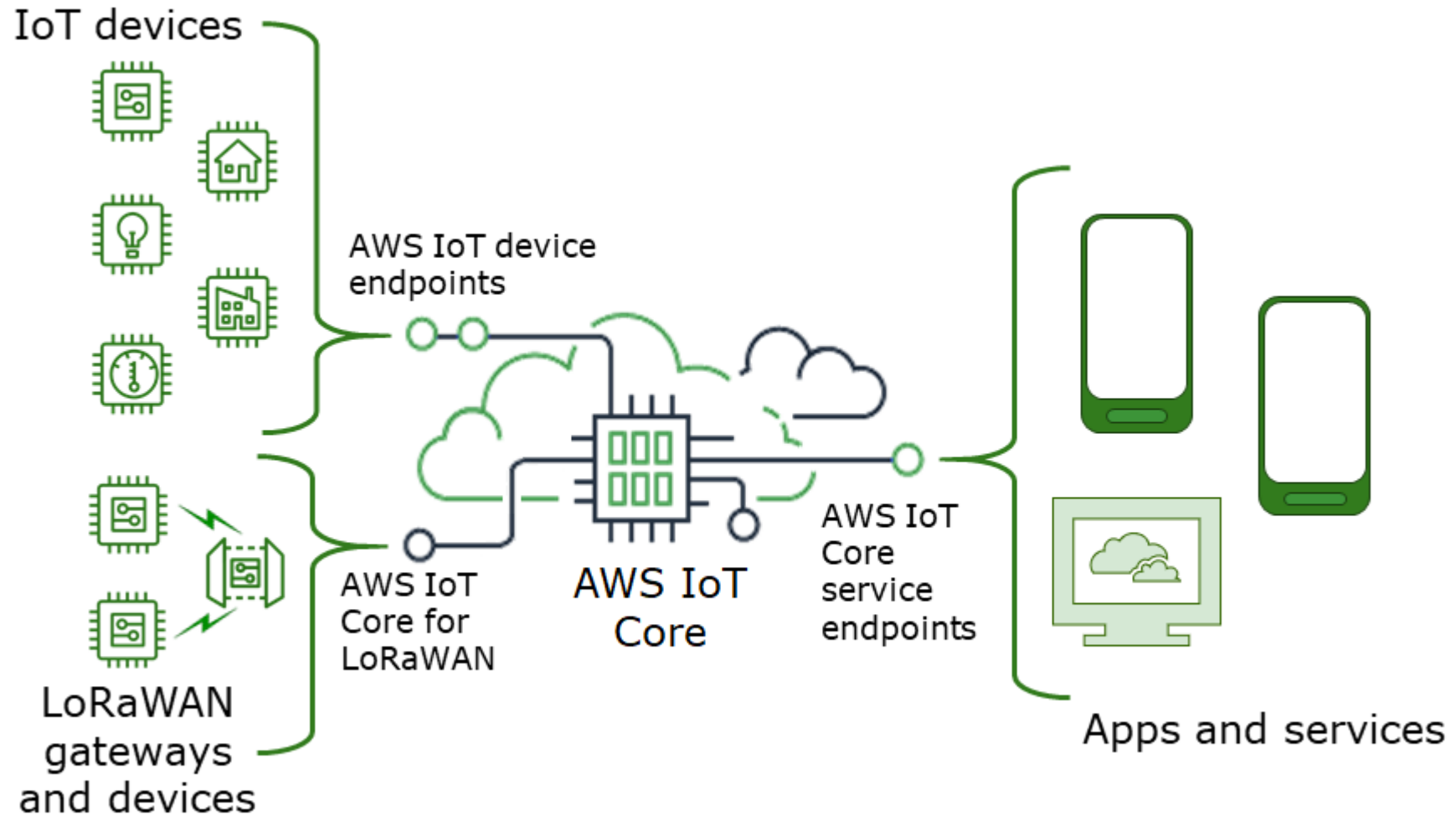


IoT Frameworks - Categories

- Overview of Major Cloud IoT Frameworks

Feature	AWS IoT Core	Azure IoT Hub
Protocols	MQTT, HTTPS, WebSocket	MQTT, AMQP, HTTPS
Device Management	Device Shadows	Device Twins
Data Processing	Rules Engine → AWS services	Time Series / Azure services
Security	IAM + X.509 certificates	SAS tokens + X.509
ML/AI	SageMaker + Greengrass	Azure ML + IoT Edge
Edge Computing	AWS Greengrass	Azure IoT Edge
Scalability	Billions of devices	Elastic scaling

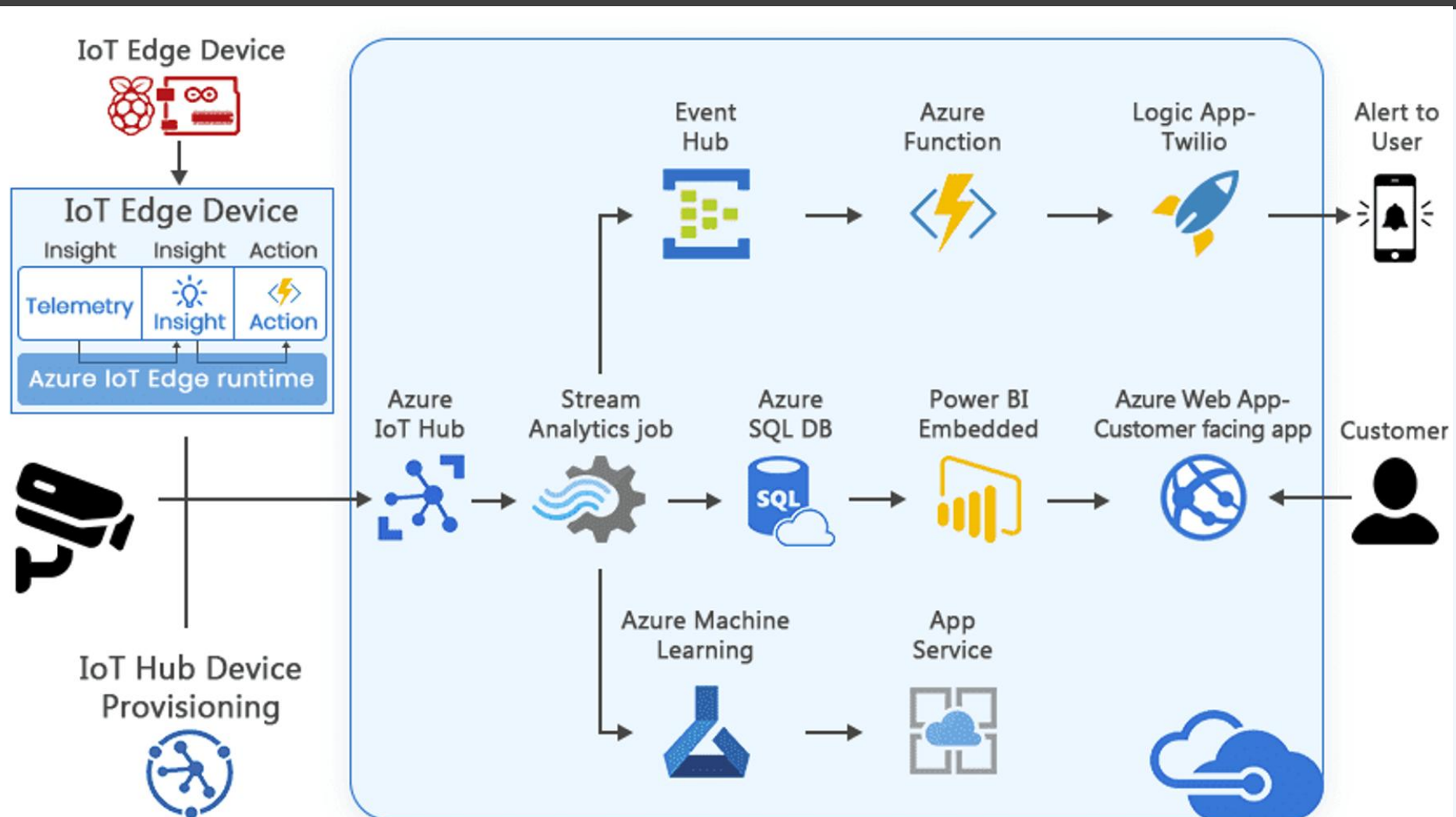
IoT Frameworks - AWS IoT Core



IoT Frameworks - AWS IoT Core

Feature	What It Does
Device Gateway	Secure two-way device–cloud communication
Device Shadow	Stores device state; enables access even when offline
Rules Engine	Filters and routes device data to AWS services
Device Registry	Stores device identity, metadata, and configuration
Security	X.509 authentication, IAM access control, TLS encryption
Greengrass	Enables local processing, ML inference, and cloud synchronization
OTA Updates	Secure over-the-air firmware updates

IoT Frameworks - Azure IoT Core



IoT Frameworks - Azure IoT Core

Feature	What It Does
Device Connectivity	Secure two-way communication via MQTT, AMQP, or HTTPS
Device Twins	Stores desired & reported device properties
Direct Methods	Remotely invokes device actions and receives responses
File Upload	Uploads firmware, sensor data, and other files to Azure Storage
Device Provisioning	Securely onboards and registers devices at scale
Security	SAS tokens, X.509 certificates, and TPM-based authentication
Edge Computing	Azure IoT Edge enables local AI, analytics, and custom processing
Integration	Integrates with Power BI, Azure ML, and other Azure services

IoT Frameworks - Open-Source

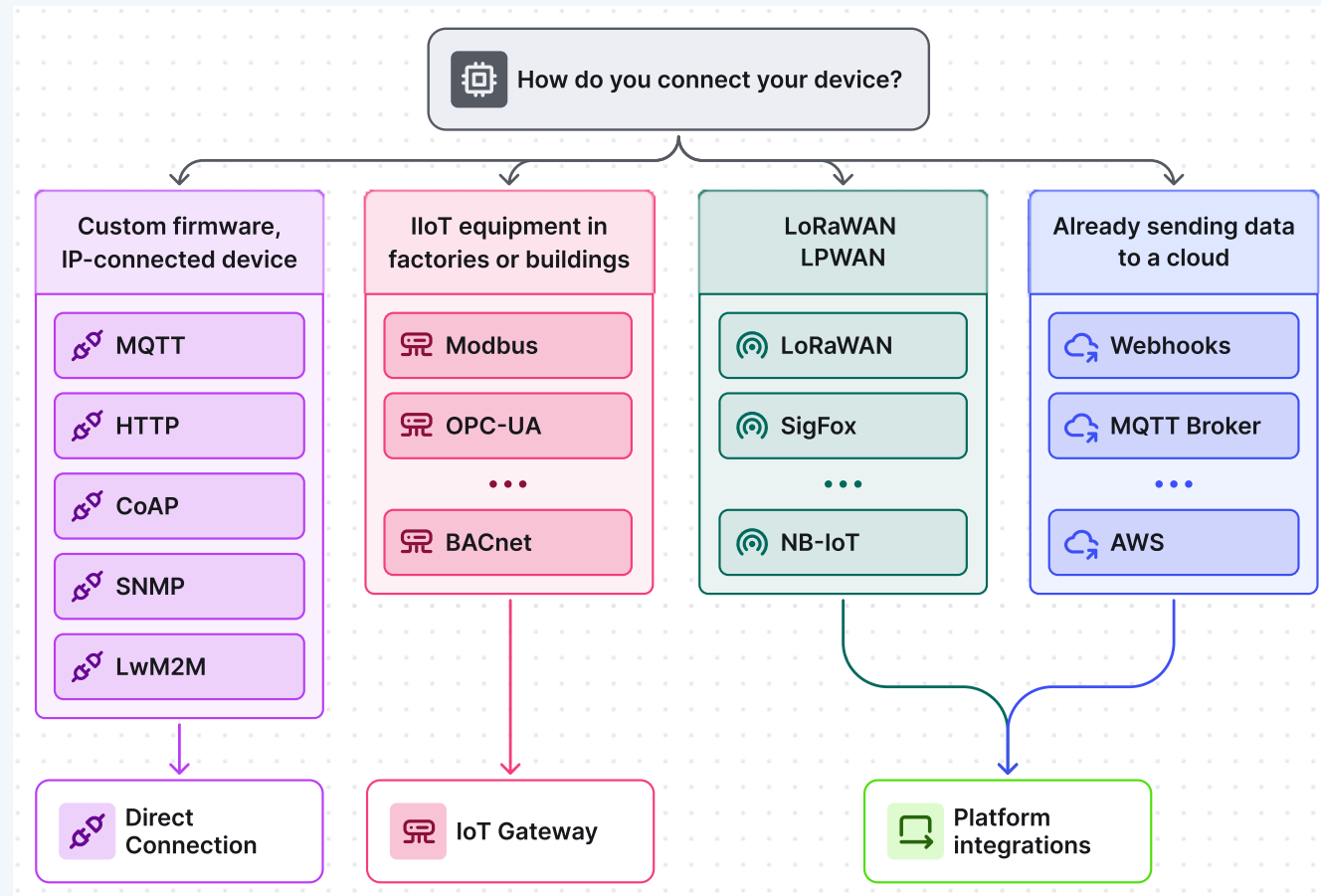
- Open-source IoT frameworks offer an alternative to cloud-based solutions by providing complete code access and self-hosting capabilities
- These frameworks appeal to organizations that require complete control over their IoT infrastructure or want to avoid vendor lock-in

IoT Frameworks - Open-Source

Framework	Type	Best Suited For
ThingsBoard	Device management and visualization platform	Custom IoT solutions requiring rich dashboards and flexible device management
Kaa IoT	Enterprise-grade IoT platform	Industrial applications, complex device fleets, and enterprise deployments
ESP RainMaker	Cloud platform specifically for ESP devices	Consumer IoT products built on ESP32, ESP32-S, and ESP32-C series
Home Assistant	Smart home automation platform	Home automation enthusiasts and smart home product development
Eclipse IoT	Collection of IoT frameworks and tools	Developers wanting to use standardized components for various IoT needs

IoT Frameworks - ThingsBoard

- Device management, data collection, processing, and visualization capabilities



Embedded Platforms for IoT Systems



Embedded Platforms for IoT Systems

- Introduction
- Platform Fundamentals
- Platform Landscape
- Selection Criteria
- Development Tools and Ecosystem
- Case Study

Embedded Platforms - Introduction

- IoT system = physical things + embedded intelligence + connectivity + cloud services
- Typical architecture:
Things/Sensors → Edge/Gateway → Cloud → Applications
- The embedded platform sits at the edge — it is the “brain” of the IoT device
- Core edge functions:
Sense → Compute → Connect → Act

What Is an Embedded Platform?

- Embedded Platform: Hardware + Software for a Dedicated Job
- A combination of hardware and software designed for a specific function
- Hardware: MCU/MPU, memory, I/O, peripherals, radios
- Software: bare metal, RTOS, or Linux
- In IoT, it converts raw signals into useful data and actions
- Examples: STM32, ESP32, BeagleBone, Raspberry Pi

Why Platform Selection Matters

- Different IoT jobs need different platforms
- Key trade-offs:
Power, real-time, compute, connectivity, cost, security
- MCU end: STM32 / ESP32 → low power, real-time, sensor nodes
- Linux SBC end: BeagleBone / Raspberry Pi → rich compute, gateways, vision
- Essential question:
When do you use an MCU, and when do you need a Linux SBC?

Platform Fundamentals - MCU vs MPU vs SBC

MCU (Microcontroller Unit)

- CPU + memory + peripherals on one chip
- Runs bare metal or RTOS
- Low power, low cost, real-time
- Examples: STM32, ESP32

MPU (Microprocessor Unit)

- CPU only; needs external memory and peripherals
- Runs full OS like Linux
- Higher compute, higher power
- Often the core of an SBC

SBC (Single Board Computer)

- Complete computer on one board
- Runs Linux, has rich I/O, networking, display
- Examples: BeagleBone, Raspberry Pi

Hardware Building Blocks of an IoT Platform

- **CPU core** : ARM Cortex-M (MCU) vs Cortex-A (Linux SBC)
- **Memory** : Flash/SRAM on MCU; DDR + eMMC/SD on SBC
- **I/O and peripherals:** GPIO, UART, I2C, SPI, ADC, PWM, CAN
- **Connectivity** : Wi-Fi, BLE, Ethernet, LoRa, cellular
- **Power subsystem** : battery, regulator, sleep modes
- **Expansion** : headers, capes, HATs, shields

Microcontroller Platforms: STM32 and ESP32

- **STM32 (STMicroelectronics)**
- ARM Cortex-M cores (M0 to M7)
- Bare metal or RTOS (FreeRTOS, Zephyr)
- Excellent real-time, low power, rich analog and timers
- Ideal for: motor control, industrial sensors, precise I/O
- Connectivity: usually external (LoRa, BLE, Ethernet)



Microcontroller Platforms: STM32 and ESP32

- **ESP32 (Espressif)**
- Dual-core Xtensa / RISC-V, up to 240 MHz
- Integrated Wi-Fi + Bluetooth/BLE
- FreeRTOS, ESP-IDF, Arduino support
- Ideal for: connected sensor nodes, small gateways, wearables
- Low cost, good compute for an MCU



Linux SBC Platforms: BeagleBone and Raspberry Pi

- **BeagleBone (Black / AI)**
- ARM Cortex-A + **PRU** real-time co-processors
- Runs Linux (Debian)
- Extensive I/O, capes for industrial expansion
- Ideal for: industrial gateways, real-time Linux control, robotics
- Strong maker and industrial community



Linux SBC Platforms: BeagleBone and Raspberry Pi

- **Raspberry Pi (4 / 5 / Zero)**
 - ARM Cortex-A, up to 8 GB RAM
 - Runs Linux (Raspberry Pi OS, Ubuntu)
 - Rich compute, USB, HDMI, camera, networking
 - Ideal for: edge gateways, vision, dashboards, prototyping
 - Huge ecosystem, but not hard real-time



Platform Comparison

Feature	STM32	ESP32	BeagleBone	Raspberry Pi
Class	MCU	MCU + radio	Linux SBC	Linux SBC
OS	Bare metal / RTOS	FreeRTOS	Linux + PRU	Linux
CPU	Cortex-M	Xtensa / RISC-V	Cortex-A + PRU	Cortex-A
Connectivity	External	Wi-Fi + BLE	Ethernet, Wi-Fi	Ethernet, Wi-Fi, BT
Real-time	Excellent	Good	Good (PRU)	Limited
Power	Very low	Low	Medium	High
Compute	Low	Medium	Medium-High	High
Best for	Control, sensors	Connected nodes	Industrial edge	Gateways, vision

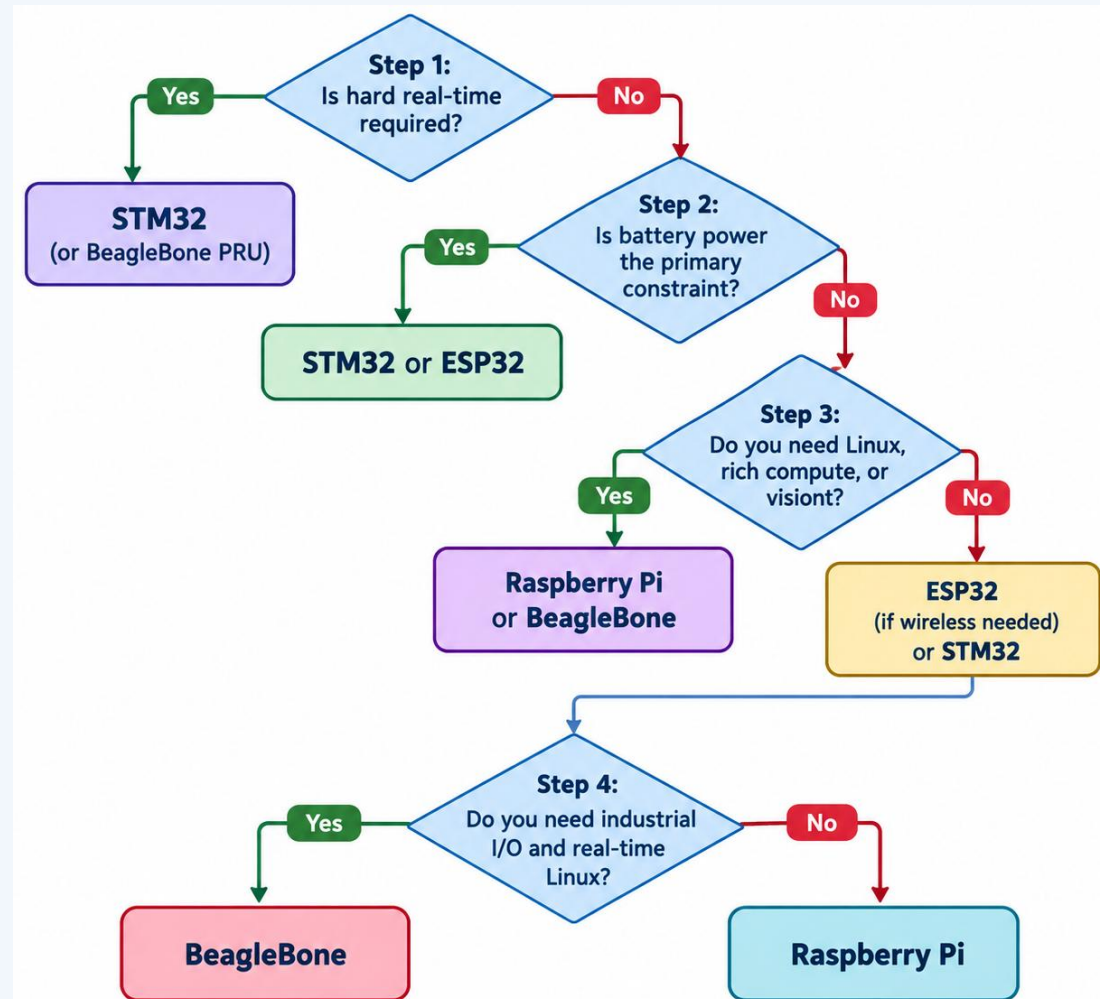
Key Selection Criteria - What Drives Platform Choice?

- **Power**
 - Battery life, sleep modes, energy harvesting
 - STM32/ESP32 excel; SBCs need mains or large battery
- **Real-time**
 - Deterministic response, jitter, latency
 - STM32 best; BeagleBone PRU good; Raspberry Pi limited
- **Compute & Memory**
 - Data processing, vision, analytics, multiple tasks
 - Raspberry Pi best; ESP32 moderate; STM32 limited
- **Connectivity**
 - Wi-Fi, BLE, LoRa, cellular, Ethernet
 - ESP32 integrated Wi-Fi/BLE; SBCs have Ethernet/Wi-Fi; STM32 often external

Key Selection Criteria - What Drives Platform Choice?

- **Cost**
 - Unit cost, development cost, ecosystem cost
- **Security**
 - Secure boot, crypto, secure storage, OTA updates
- **Ecosystem**
 - Tools, libraries, community, industrial support

Decision Framework



Matching Use Cases to Platforms

Use Case	Recommended Platform	Why
Battery soil sensor, long range	ESP32 + LoRa or STM32 + LoRa	Low power, external radio
Motor control, microsecond timing	STM32	Deterministic real-time
Smart home gateway, MQTT, dashboard	Raspberry Pi	Linux, networking, compute
Industrial machine control, Linux + precise I/O	BeagleBone	PRU for real-time, capes
Vision-based edge node	Raspberry Pi	GPU, camera, compute
Cheap BLE wearable	ESP32 or STM32 + BLE	Low power, small, connected